Team Name : QuantEdge

Team members Names	**Engineerings Specialisation**	Graduating year	College Names
Mohd Sahil	Chemical engineering	2027	Delhi Technological University
Priyansh	Chemical engineering	2027	Delhi Technological University

	Mohd Sahil	**Priyansh**
Any projects you've worked on	**Sales Analytics ETL System** Engineered data pipeline to clean, structure, and analyze sales data, delivering business-ready insights.	**ARIS**: AI Riding Integrity System
Past Hackathon experiences	**UIDAI Hackathon (Data Engineering Lead)** Designed ETL pipeline, processed large datasets & enabled predictive, insight-driven decision system	N/A

Problem Statement 3:

Accurate calculation of a driving behavior score on a two-wheeler

Overview of our Solution

Our solution is a real-time rider behavior monitoring system designed to improve road safety for two-wheeler riders. It uses sensors such as IMU, GPS, and side proximity sensors to collect live ride data like speed, acceleration, braking, turning, and nearby vehicle movement.

This data is processed using a Python-based scoring model, which calculates a rider safety score out of 100 and classifies the rider as safe, moderate risk, or unsafe..

Process Flow

1. **Sensors Layer**
   - IMU
   - GPS
   - Proximity Sensor
2. **Data Acquisition Layer**
   - Real-time Capture
   - Noise Filtering
3. **Database Layer**
   - SQL Logging
   - Ride History
4. **Analytics Layer**
   - Python Scoring
   - Event Detection
5. **Visualization + Alerts**
   - Rider Score
   - LED Alerts
   - Dashboard

Uniqueness of our Solution

01 Real-time rider scoring instead of only speed monitoring

02 Side-aware helmet LED alerts for left-right vehicle proximity

03 Data analytics integration using Python and SQL

04 GPS-based risk zone identification for unsafe routes

05 Cost-effective and scalable prototype, can also work using smartphone sensors

Design Details

Market Research and Literature Survey

Road accidents involving two-wheelers are very common in India, especially in dense traffic and peak hours. Current monitoring systems mainly focus only on speed, which does not fully represent rider behavior. There is growing demand in insurance, fleet management, and rider safety systems for behavior-based analytics.

Working Principle

The system collects live sensor data and checks whether the rider is braking harshly, overspeeding, turning aggressively, or riding too close to nearby vehicles.
Unsafe events reduce the score based on a penalty model.

Prototype Sample

A Python prototype was developed to simulate the scoring logic.

Sample output:

- Rider Score = 73
- Risk Level = Moderate Risk
- Left LED = Red
- Right LED = Green

rider_score.py > ...

```python
# Sample ride data
harsh_brakes = 2
sudden_acceleration = 1
overspeed_events = 1
unsafe_turns = 1
proximity_alerts = 2

# Initial score
score = 100

# Penalty calculation
score -= 5 * harsh_brakes
score -= 4 * sudden_acceleration
score -= 6 * overspeed_events
score -= 3 * unsafe_turns
score -= 2 * proximity_alerts

# Risk level classification
if score >= 85:
    risk = "Safe Rider"
elif score >= 70:
    risk = "Moderate Risk"
else:
    risk = "Unsafe Rider"

# LED logic
left_led = "RED"
right_led = "GREEN"

# Output
print("Rider Score:", score)
print("Risk Level:", risk)
print("Left LED:", left_led)
print("Right LED:", right_led)
```

Benchmarking

Existing systems: speed-only monitoring
Our solution: multi-parameter behavior analysis + alerts

Innovation

- rider score analytics
- real-time helmet alerts
- route risk mapping

Driving Behavior Score Calculation

The rider starts with an initial score of 100.
Penalty points are deducted for every unsafe riding event detected in real time.

S=100-(5B+4A+6O+3T+2P)

Where:

- B = harsh braking events
- A = sudden acceleration events
- O = overspeed events
- T = unsafe turns
- P = proximity alerts

Sample Ride Data

```
# Sample ride data
harsh_brakes = 4
sudden_acceleration = 1
overspeed_events = 1
unsafe_turns = 3
proximity_alerts = 2
```

Output

```
r_score.py"
Rider Score: 57
Risk Level: Unsafe Rider
Left LED: RED
Right LED: GREEN
PS C:\Users\Sahil\OneDrive\Desktop\PYTHON RIDE>
```

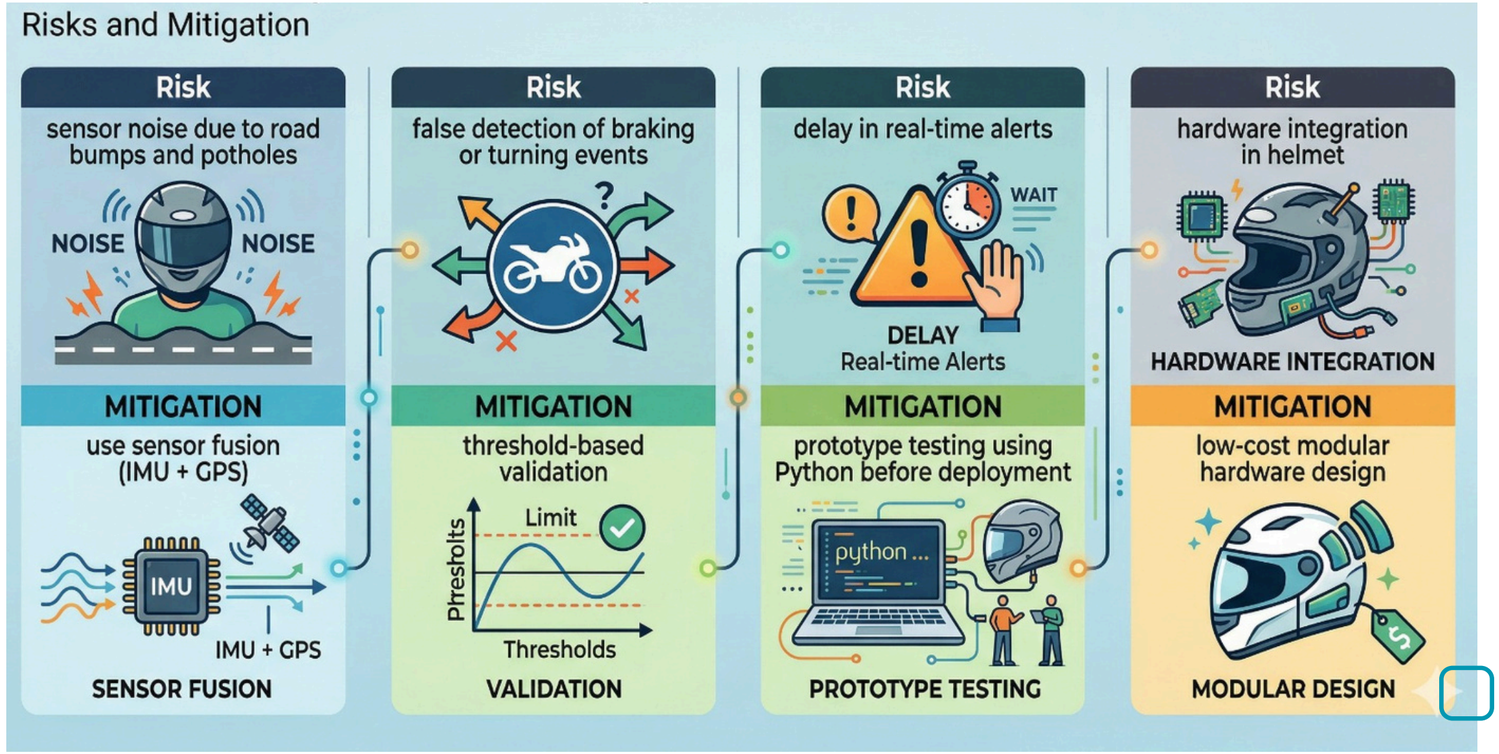
Major Risk and Mitigation

Risk

- sensor noise due to road bumps and potholes
- false detection of braking or turning events
- delay in real-time alerts
- hardware integration in helmet

Mitigation

- use sensor fusion (IMU + GPS)
- threshold-based validation
- prototype testing using Python before deployment
- low-cost modular hardware design



Business Potential / Market Demand

Market Potential

India has one of the largest two-wheeler markets in the world, with increasing road safety concerns. There is strong demand for data-driven safety and telematics solutions, especially in insurance and fleet sectors.

Business Potential

This solution has strong use cases in multiple sectors.

- Insurance companies can use rider scores for fair premium pricing and safe rider rewards
- delivery and logistics companies can monitor rider safety and training needs
- driving schools can use it for performance evaluation
- smart mobility companies can integrate it into their rider safety platforms

Milestones / Timeline / Budget

Major Milestones and Timelines

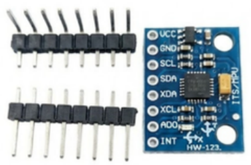
Major Milestones and Timelines

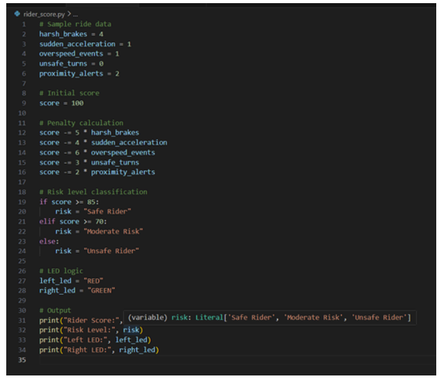
Week 1: concept design and research
Week 2: Python scoring prototype
Week 3: dashboard + SQL data structure
Week 4: sensor integration and testing
Week 5: final validation and presentation

Budget for Prototype

Budget for Prototype

IMU sensor module – ₹500–700
GPS module – ₹700–1000
proximity sensors – ₹300–500
LED setup – ₹200
software tools – Python / SQL (open source)





```
rider_score.py > ...
# Sample ride data
harsh_brakes = 4
sudden_acceleration = 1
overspeed_events = 1
unsafe_turns = 0
proximity_alerts = 2

# Initial score
score = 100

# Penalty calculation
score -= 5 * harsh_brakes
score -= 4 * sudden_acceleration
score -= 6 * overspeed_events
score -= 3 * unsafe_turns
score -= 2 * proximity_alerts

# Risk level classification
if score >= 85:
    risk = "Safe Rider"
elif score >= 70:
    risk = "Moderate Risk"
else:
    risk = "Unsafe Rider"

# LED logic
left_led = "RED"
right_led = "GREEN"

# Output
print("Rider Score:", (variable) risk: Literal['Safe Rider', 'Moderate Risk', 'Unsafe Rider']
print("Risk Level:", risk)
print("Left LED:", left_led)
print("Right LED:", right_led)
```

Estimated Prototype Cost: ₹2000 – ₹2500

**The estimated prototype cost is ₹2,000–₹2,500; however, the per-unit cost can be significantly reduced under bulk manufacturing and large-scale deployment.